capital

Custom Constraint Development

Software Version 2015.1

February 2016

© 2016 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**

Interfaces **9**

Attributes and Properties – Length Change for Jumper 9
Attributes and Properties – Length Change for Wire 9
Attributes and Properties – Multicore Specification 10
Attributes and Properties – Set Integrator Attribute/Property on Object 10
Attributes and Properties – Splice Specification 11
Attributes and Properties – Terminal Type Specification 12
Attributes and Properties – Wire Specification 14
Placement – Assign Connector by Attribute/Property 15
Placement – Map to same cavity by signal Attribute/Property 15
Placement – Option place by Attribute/Property 16
Placement – Place by Attribute/Property 16
Ground Refinement – Ground device combination 17
Ground Refinement – Max Ground Slot Connector Cavities 18
Ground Refinement – max multitermed Ground Signals 18
Ground Refinement – Multiterm Ground Devices 19
Routing – Backshell Termination by Attribute/Property 19
Routing – Cost of Multiterm 20
Routing – Cost of Splice 21
Routing – Cost of Wire per Unit Length 22
Routing – Cost per wire for Inline/Junction Box 23
Routing – Generate center strip splices 23
Routing – Map to Inline Cavity by Signal/Attribute Property 24
Routing – Max Wires per Multiterm 25
Routing – Max Wires per Splice 26
Routing – Minimum Splice Separation 27
Routing – Network Specification 28
Routing – Pigtails Usage by Attribute/Property 29
Routing – Protect Wiring by Attribute/Property 29
Routing – Route by Attribute/Property 30
Module Assignation – Assign Module Codes for All Design Objects via Plugin 31
Module Assignation – Assign Module Codes for Selected Object(s) 34
Module Assignation – Assign Technical Module Codes for Selected Object(s) 35
Cavity Details Generation – Set Cavity Details Generation Method 37

Exercise **39**

Guidance **50**

Examples **51**

Introduction

This document describes the process for developing Custom Constraints plugins for Capital applications like Capital Integrator or Capital ModularXC. This allows you to apply your own placements and synthesis rules to the designs you create. If you are **not** interested in developing Custom Constraints, you do not have to read this document. If you are interested in developing Custom Constraints, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

For Capital Integrator, there are several types of custom constraint that can be implemented. They fall into the following categories:

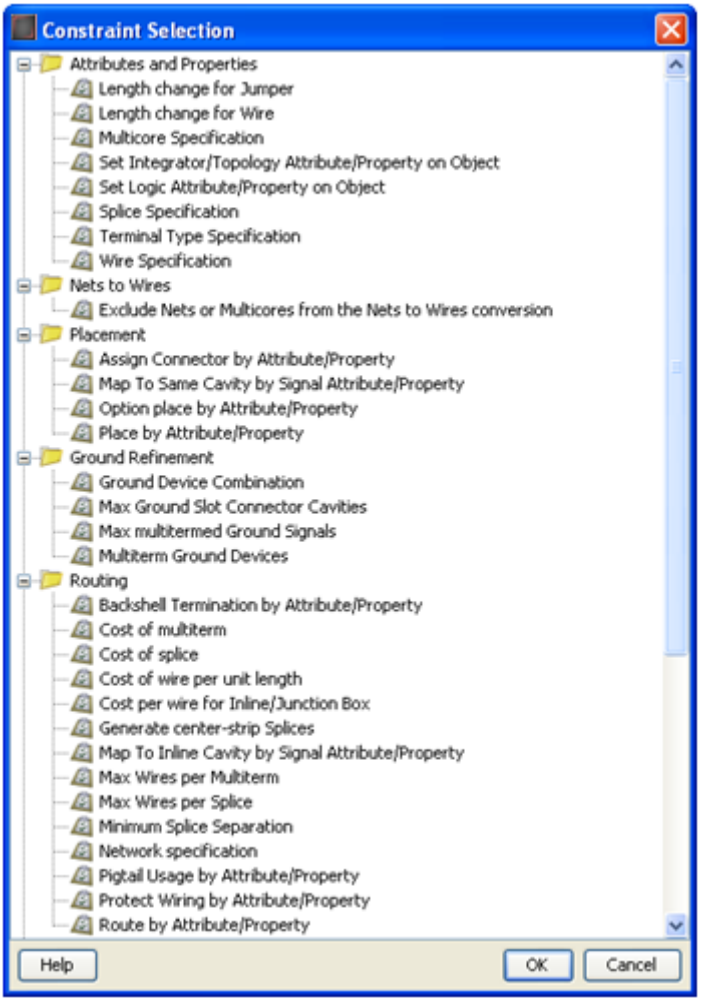
- **Attributes and Properties** – these constraints allow additional attributes or properties to be added to certain objects to set their specification.

- **Placement** – these constraints control which devices are placed in which slots.

- **Routing** – these constraints control the way in which signals are routed and how they are terminated.

- **Ground Refinement –** these constraints control the way how grounds are terminated or combined.

In Capital Integrator, if you click on the Edit -> Rules menu. Click on the [New…] button to create a new rule. Give the rule a name then click [OK]. Ensure your new rule is selected, then click on the [Add Constraint…] button, you will be able to see these categories in the following dialog:



You will be able to define your own custom constraint for each item shown above. This is achieved by implementing a Java interface that has been defined for each one.

Each of these Java interfaces will have one or more `match()` methods. Each of the match methods returns a Boolean value; you should return true if the constraint is to be applied, false if not.

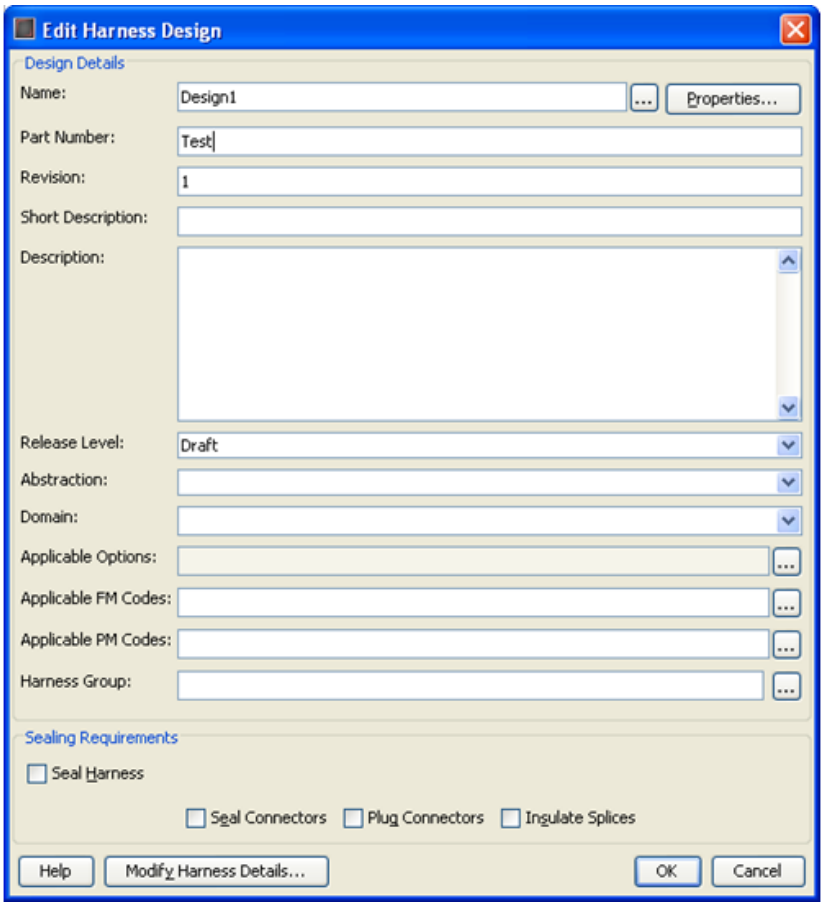
One or more business objects will also passed into the `match()` methods as input parameters. Based on the information in these objects, you can decide if the constraint should be applied or not (whether you return true or false from the method). The type and number of business objects passed in will vary depending on the type of constraint being implemented.

A result object is also passed into the `match()` method. These result objects allow you to pass back additional information (they are essentially in-out parameters). For those constraints that are setting attributes on objects, for example, a result object is passed in that is capable of storing the attributes to set. When you implement the `match()` method, you will need to populate the result object if the constraint is to be applied (and then return true).

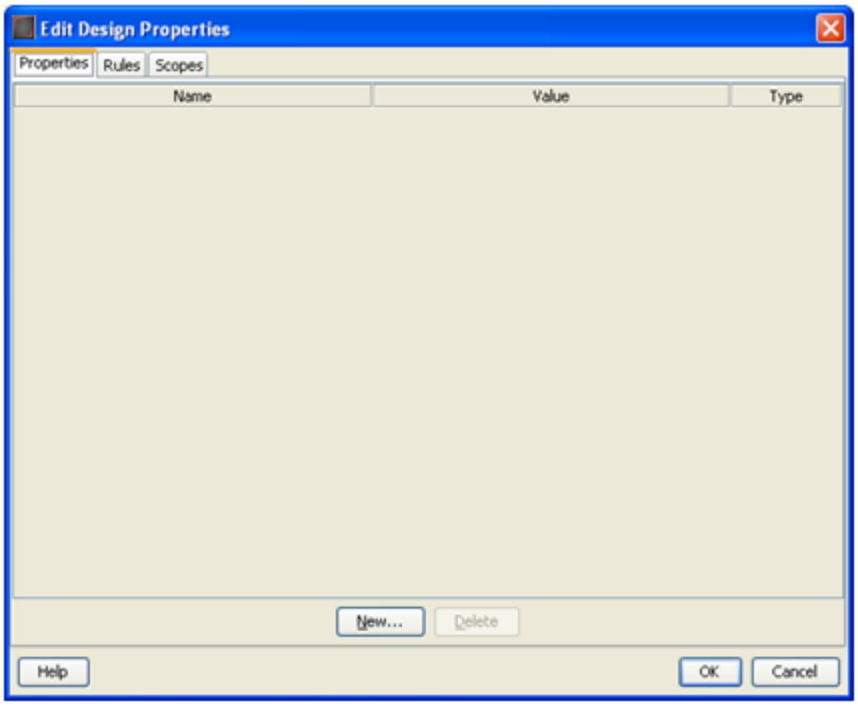
In addition to the methods that are specific to custom constraints, you will need to implement the `getName()`, `getDescription()` and `getVersion()` methods that all plugins need to implement.

For Capital ModularXC, custom constraints allow you to implement your own module code assignation routines to augment or replace the built-in assignation routines. Normally when there are no constraints, if you select module code assignation a dialog will be displayed that allows you to control the built-in assignation routines. However, if you add constraints to a design, those constraints will control assignation and you will not see the dialog.

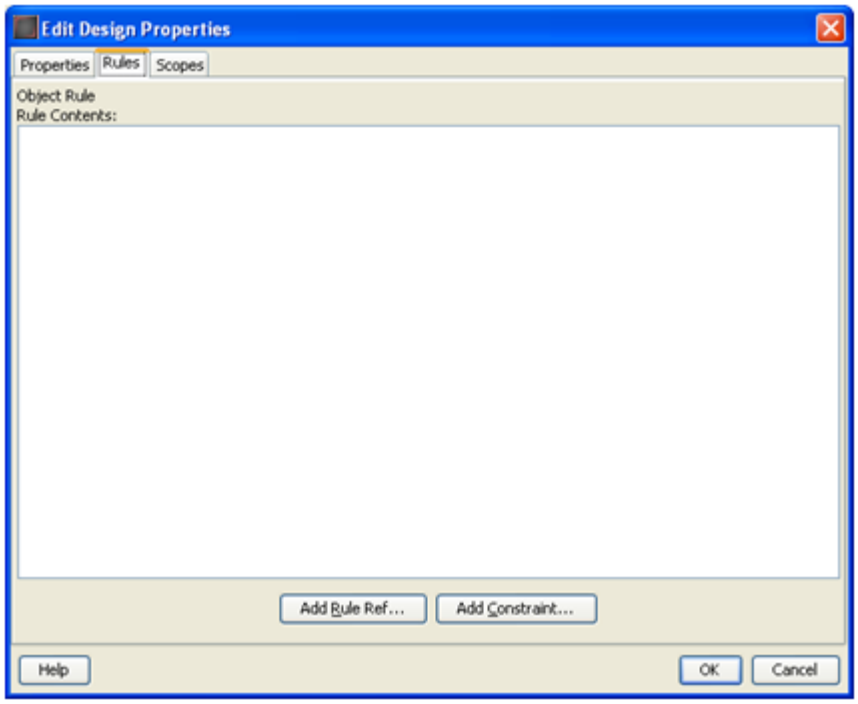
To add these constraints, you will need to open the edit dialog for the harness design from Capital Project or Capital ModularXC



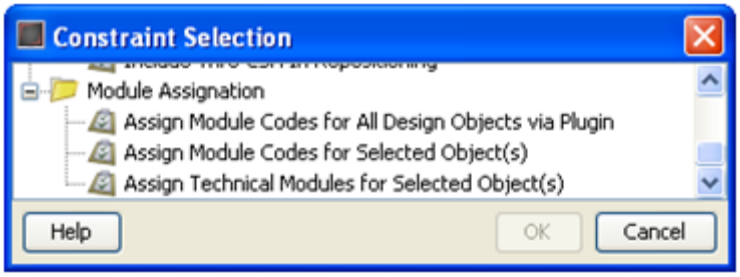
Next click on the [Properties…] button



Then click on the [Rules…] tab



Now, if you click on the [Add Constraint…] button you will see the following module assignation constraints.



As you can see, there are three types of constraint for module assignation. There are also two types of module code: functional and production (See Capital ModularXC documentation for more information). This means there are six types of constraints available in total.

- **Assign Module Codes for All Design Objects via Plugin**

  These constraints allow you to implement your own assignation routine that does multiple object assignations. You can, using these constraints, assign module codes to a number of objects of different types in an order determined by your algorithm.

  The purpose of this constraint is to allow you to completely replace the built-in assignation routines in Capital ModularXC with your own. As a result, adding this constraint will prevent the other types of constraint from being applied. This constraint will have to do all the work.

  For functional codes, you will need to create a plugin that implements the `IXFunctionalModuleAssignationConstraint` interface.

  For production codes, you will need to create a plugin that implements the `IXProductionModuleAssignationConstraint` interface.

- **Assign Module Codes for Selected Object(s)**

  This constraint allows you apply the built-in assignation routine to various object-types with various options. Once added, you can override the built-in behavior with your own custom assignation routine for particular types of object.

  This allows you to combine the built-in standard assignation routines with your own assignation routines. For example, if the built-in standard assignation routines are fine for all types of object except splices, you can add all the constraints for the various types object including splices. Then you can override the algorithm just for splices with a custom constraint.

  If the built-in or custom assignation fails, then technical assignation will be attempted if a constraint for this has been added.

If you do not want to do standard assignation for a particular type of object, then do not add a constraint for it.

For functional codes, you will need to create a plugin that implements the `IXFunctionalModuleAssignationObjectAlgorithmConstraint` interface.

For production codes, you will need to create a plugin that implements the `IXProductionModuleAssignationObjectAlgorithmConstraint` interface.

- **Assign Technical Module Codes for Selected Object(s)**

  This constraint allows you apply the built-in technical assignation routine to various object-types. Once added, you can override the built-in behavior with your own custom assignation routine.

  Again, this allows you to combine the built-in technical assignation routines with your own assignation routines. For example, if the built-in technical assignation routines are fine for all types of object except splices, you can add all the constraints for the various types object including splices. Then you can override the algorithm just for splices with a custom constraint.

  Technical assignation will be attempted if the standard built-in assignation or custom constraint fails.

  If you do not want to do technical assignation for a particular type of object, then do not add a constraint for it.

  For functional codes, you will need to create a plugin that implements the `IXFunctionalModuleAssignationTechnicalConstraint` interface.

  For production codes, you will need to create a plugin that implements the `IXProductionModuleAssignationTechnicalConstraint` interface.

We shall now look at the most common constraints that are available to you. It should be noted that what follows is not a list of all the types of custom constraint that you can implement. Please look at the following document in the installation directory:

```
doc/plugin/api/PluginObjectsClassDiagram.png
```

If you look at all the children of the `IXConstraint` interface, you will find all the constraints that you can implement.

Interfaces

Attributes and Properties – Length Change for Jumper

NOTE: *This constraint is not available in Capital Systems.*

Custom Constraint allows the length of a jumper to be controlled during routing. The result will be used to update the Length Change attribute of the wire.

```
public interface IXAdditionalWireLengthConstraint
{
     boolean match(IXWire jumperwire,
                   IXWireLengthResult result)
}
```

Determines if constraint applies for the given wire. If so, allows the length change values to be specified.

Parameters:
`jumperwire` - to which length changes should be applied
`result` - Use to store the length change values to be applied on to the wire, when match returns "true".

Returns:
`boolean` - Does the constraint apply to the wire passed in .

Attributes and Properties – Length Change for Wire

NOTE: *This constraint is not available in Capital Systems.*

Custom Constraint allows the length of a wire to be controlled during routing. The result will be used to update the Length Change attribute of the wire.

```
public interface IXAdditionalWireLengthConstraint
{
     boolean match(IXWire wire,
                   IXWireLengthResult result)
}
```

Determines if constraint applies for the given wire. If so, allows the length change values to be specified.

Parameters:
`wire` - to which length changes should be applied

`result` - Use to store the length change values to be applied on to the wire, when match returns "true".

Returns:
`boolean` - Does the constraint apply to the wire passed in .

Attributes and Properties – Multicore Specification

The "Multicore Specification" custom constraint allows you to set attributes or properties on the physical Multicores created. Here is the interface:

```
public interface IXMulticoreAttributeConstraint extends
IXConstraint
{
     boolean match(IXShield           shield,
                   IXMulticore        multicore,
                   IXHarness          harness,
                   IXAttributesResult result);
}
```

The `IXShield` input parameter allows you to obtain information about the shield being used for the multicore.

The `IXMulticore` input parameter allows you to obtain information about the multicore. This is the multicore upon which any attributes or properties will be set (if you return true). You cannot set these attributes or properties directly on this object. You must use the `IXAttributesResult` object and let Capital Integrator do this for you (more later).

The `IXHarness` input parameter allows you to obtain information about the harness the multicore is in.

If you decide that the constraint should be applied, the `match()` method should return true, and you will need to populate the result object with the attributes that need to be set. The result object passed in will implement the `IXAttributesResult` interface.

Using this interface, you can use the `addAttribute()` or the `addProperty()` methods to add attribute or property changes. When you return true from the `match()` method, the properties and attributes you have added to the `IXAttributesResult` object will be applied. Any existing values will be replaced with new ones. Any properties that were not there before will be added.

Attributes and Properties – Set Integrator Attribute/Property on Object

Custom Constraint - Used to set attributes and properties on objects that match the constraint.

```
public interface IXSetAttributeConstraint extends
IXConstraint
  {
        boolean match(     IXObject obj,
                           IXAttributesResult result)
  }
```

Determine if the specified object matches the constraint. If so, set the result object with the corresponding changes that are defined in the constraint.

Parameters:
`obj` - The object to match against.
`result` - Stores the changes that would be applied to attributes and properties of the specified object as a result of applying this constraint. These changes are only processed if the constraint matches.

Returns:
`true` if the constraint matches the object, `false` otherwise.

Attributes and Properties – Splice Specification

The "Splice Specification" custom constraint allows you to set attributes or properties on splices. Here is the interface (note that there are two `match ()` methods that need to be implemented):

```
public interface IXSpliceAttributeConstraint extends
IXConstraint
{
     boolean match (IXSplice           splice,
                    IXSignal           signal,
                    IXBundle           bundle,
                    IXAttributesResult result);

     boolean match (IXSplice           splice,
                    IXSignal           signal,
                    IXBundleRegion     region,
                    IXAttributesResult result);
}
```

Again, if you want this constraint to be applied, given the input parameters, you will need to return true.

The `IXSplice` input parameter allows you to obtain information about the splice. This is the splice upon which any attributes or properties will be set (if you return true).

The `IXSignal` input parameter allows you to obtain information about the signal that was routed via the splice.

For one of the methods, the `IXBundle` input parameter allows you to obtain information about the bundle the splice is in.

For the other method, the `IXBundleRegion` input parameter allows you to obtain information about the bundle region the splice is in.

You do not have to implement both methods. If you only wish to implement just one method, simply ensure that the other method always returns false.

If you decide that the constraint should be applied, you will need to populate the result object with the attributes that need to be set. The result object passed in will implement the `IXAttributesResult` interface.

Using this interface, you can use the `addAttribute ()` or the `addProperty ()` methods to add attribute or property changes. When you return true from the `match ()` method, the properties and attributes you have added to the `IXAttributesResult` object will be applied. Any existing values will be replaced with new ones. Any properties that were not there before will be added.

Attributes and Properties – Terminal Type Specification

The "Terminal Type Specification" custom constraint allows you to set one of the following properties: `StartTerminalSpec` or `EndTerminalSpec` that are connected to the matched signal. So when Capital Integrator decides to connect a wire to a cavity or a backshell termination, this information is used to set the specification of that termination.

Here is the interface:

```
public interface IXWireTerminalSpecConstraint extends
IXConstraint
{
     boolean match (IXWireEnd     wireEnd,
                    IXSignal      signal,
                    IXCavity      cavity,
                    IXValueResult result);

     boolean match (IXWireEnd                wireEnd,
                    IXSignal                 signal,
                    IXBackshellTermination backshellTerm,
                    IXValueResult            result);
}
```

As you can see, there are two `match ()` methods. When implementing these methods, you should return false if you do not want the constraint to apply, or true if you want the constraint to apply. The decision to apply the constraint should be made, by you, based on some aspect of the parameters being passed in.

Again, you do not have to implement both methods. If you only wish to implement just one method, simply ensure that the other method always returns false.

The `IXWireEnd` input parameter is, at present, only a place holder. In a future release, you will be able to use this parameter to obtain information about the wire end being terminated so you can make a more informed decision.

The `IXSignal` input parameter allows you to obtain information about the signal that was routed here.

The `IXCavity` input parameter is present on one of the `match ()` methods. If the signal is being terminated on a cavity, it is this method that will be used. By passing in the `IXCavity` as a parameter, it allows you to adjust the terminal specification based on some property or attribute of the cavity.

The `IXBackshellTermination` input parameter is present on one of the `match ()` methods. If the signal is being terminated on a backshell termination, it is this method that will be called by Capital Integrator. By passing in the `IXBackshellTermination` as a parameter, it allows you to adjust the terminal specification based on some property or attribute of the backshell termination.

If you decide that the constraint should apply, you will need to set the wire terminal specification. A result object is passed in to the `match ()` method for this task. Here is the interface.

```
public interface IXValueResult
{
     void setValue (String value);

     String getValue ();
}
```

You will need to use the `setValue ()` method on this object to set the value of the terminal specification, then you will need to return true when you exit the `match ()` method.
The decision on whether `StartTerminalSpec` or `EndTerminalSpec` will be set is made by Capital Integrator.

Attributes and Properties – Wire Specification

The "Wire Attribute Specification" custom constraint allows you to set any of the attributes or properties on the wires created when routing a signal. Here is the interface:

```
public interface IXWireAttributeConstraint extends
IXConstraint
{
     boolean match (IXWire              wire,
                    IXSignal           signal,
                    IXHarness          harness,
                    IXAttributesResult result);

}
```

The `IXWire` input parameter allows you to obtain information about the wire that was routed. This is also the wire upon which any attributes or properties will be set. You cannot set these attributes or properties directly on this object. You must use the `IXAttributesResult` result object.

The `IXSignal` input parameter allows you to obtain information about the signal that was routed via the wire.

The `IXHarness` input parameter allows you to obtain information about the harness the wire is in.

If you decide the constraint should be applied, you will need to populate the result object with the attributes that need to be set and return true. These attributes may include: Material, Color, Wire Spec, etc. The result object passed in will implement the `IXAttributesResult` interface:

```
public interface IXAttributesResult
{
     void addAttribute (String name, String value);

     void addProperty (String name, String value);

     Collection<IXValue> getAttributes ();

     Collection<IXValue> getProperties ();
}
```

You can use the `addAttribute ()` or `addProperty()` methods to add attributes or properties to be changed. If you return true from the `match()` method, the properties and attributes you have added to the `IXAttributesResult` object will be applied to the wire. Any existing values will be replaced with new ones. Any properties that were not present before will be added.

Placement – Assign Connector by Attribute/Property

Custom Constraint - used to assign a connector of a logical device to a virtual connector of a slot during place device actions for footprint devices

```
public interface IXAssignConnectorConstraint
{
      boolean match(
      IXDeviceConnector deviceConnector,
      IXSlot slot,
      IXAbstractConnector virtualConnector,
      IXDoDontResult result)

      boolean match(
      IXLibraryConnector libConnector,
      IXSlot slot,
      IXAbstractConnector virtualConnector,
      IXDoDontResult result)

}
```

Determine if constraint applies given objects. If so, return a DO or DONT directive in the result to indicate the proper action. This is for device based footprint.

Parameters:
`deviceConnector` - Footprint Device Connector of the device to be placed.
`slot` - Where the device will be placed
`virtualConnector` - Possible Virtual Connector on the placement slot that the device connector can be assigned to.
`result` - Return enumerated type for "DO" or "DON'T". The result is set when returning "true".
`libConnector` - Harness connector defined in footprint for the device to be placed.
`slot` - Where the device will be placed
`virtualConnector` - Possible Virtual Connector on the placement slot that the connector can be assigned to.
`result` - Return enumerated type for "DO" or "DON'T". The result is set when returning "true".

Returns:
true, if the constraint apply to the objects.

Placement – Map to same cavity by signal Attribute/Property

Custom Constraint used for re-using a cavity by multiple pins with the same signal for device placement in a slot.

```
public interface IXMapToSameCavityBySignalConstraint
```

```
{
        boolean match(IXSlot slot,
                IXSignal signal,
                IXDoDontResult result)
}
```

Determine if constraint applies given objects. If so, return a DO or DONT directive in the result to indicate the proper action.

Parameters:
`slot` - Where the device will be placed.
`signal` - Signal being dealt with.
`result` - Return enumerated type for "DO" or "DON'T". The result is set when returning "true".
Returns:
`true`, if the constraint apply to the objects.

Placement – Option place by Attribute/Property

Custom Constraint - used to control device placement in slots places device with option/variant expressions

```
public interface IXOptionPlacementConstraint
{
        boolean match(
                IXDevice device,
                        IXSlot slot,
                        IXOptionExpressionResult result)
}
```

Determine if constraint applies given these objects. If so, return a valid Option Expression to be placed with the device.

Parameters:
`device` - - To be placed
`slot` - - Being considered for placement
`result` - - Return a valid option expression.
Returns:
`boolean` - Does the constraint apply to this specific set of objects?

Placement – Place by Attribute/Property

The “Placement” custom constraint allows you to develop your own rules for placement of devices into slots. Here is the interface:

```
public interface IXPlacementConstraint extends IXConstraint
{
     boolean match(IXDevice       device,
                   IXSlot         slot,
                   IXDoDontResult result);
}
```

The `IXDevice` input parameter is the device that is being considered for placement.

The `IXSlot` input parameter is the slot that this device “could” be placed into.

The objective of the `match()` method is to decide if the provided device should be placed in the specified slot (or if the device should NEVER be placed in the specified slot).

If you have come to a decision, then your `match()` method needs to return true.

In addition, you will need to confirm what type of decision you have made using the result object that was passed in. This result object implements the following interface:

```
public interface IXDoDontResult
{
     enum DoDont {DO, DONT}

     void setValue(DoDont value);

     DoDont getValue();
}
```

By calling the `setValue(DO)` on this result object, you will be indicating that the device should be placed in the specified slot.

By calling the `setValue(DONT)` on this result object, you will be indicating that the device should NOT be placed in the specified slot.

Ground Refinement – Ground device combination

Custom Constraint - used to specify ground device combination in ground slots during a "Combine All Grounds" action

```
public interface IXGroundCombinationConstraint
{

boolean match(
      java.util.Collection<IXGround> grounds,
      IXHarness harness,
```

```
        IXSlot slot,
        IXCombineDeviceResult result)
}
```

Determine if constraint applies given these objects. If so, returns in 'result' sets of ground devices to be combined

Parameters:
`grounds` - - found in the given harness of the given slot
`harness` - - a specific harness on the slot the ground signals of the ground devices (first argument) are all in this harness
`slot` - - the ground slot where the devices are placed
`result` - - client shall fill in combination data when returning "true"

Returns:
`boolean` - true if the constraint should apply to this set of objects

Ground Refinement – Max Ground Slot Connector Cavities

Custom Constraint - used to control the number of cavities in the connectors of ground slots

```
public interface IXMaxCavitiesPerGroundConnectorConstraint
{
        boolean match(IXSlot slot,
                IXCountResult result)
}
```

Determine if constraint applies given these objects. If so, returns in 'result' the max number of cavities on connectors on this slot. Return values less than 1 are invalid and will default to 1.

Parameters:
`slot` - - a ground slot
`result` - - return the number of cavites allowed on connectors of this slot

Returns:
`true` - if this constraint should be applied to this slot

Ground Refinement – max multitermed Ground Signals

Custom Constraint - used to control the number of cavities in the connectors of ground slots

```
public interface IXMaxCavitiesPerGroundConnectorConstraint
{
        boolean match(IXSlot slot,
                IXCountResult result)
}
```

Determine if constraint applies given these objects. If so, returns in 'result' the max number of cavities on connectors on this slot. Return values less than 1 are invalid and will default to 1.

Parameters:
`slot` - - a ground slot
`result` - - return the number of cavites allowed on connectors of this slot

Returns:
`true` - if this constraint should be applied to this slot

Ground Refinement – Multiterm Ground Devices

Custom Constraint - used to control the number of cavities in the connectors of ground slots during a "Combine All Grounds" action

```
public interface IXMultitermGroundDevicesConstraint
{

  boolean match(
      java.util.Collection<IXGround> grounds,
      IXHarness harness,
      IXSlot slot,
      IXMultitermSignalResult result)
}
```

Determine if constraint applies given these objects. If so, returns in 'result' sets of ground devices that will have their signals multi-termed on a common cavity.

Parameters:
`grounds` - - found in the given harness of the given slot
`harness` - - a specific harness on the slot the ground signals of the ground devices (first argument) are all in this harness
`slot` - - the ground slot where the devices are placed
`result` - - client shall fill in combination data when returning "true"

Returns:
`boolean` - true if the constraint should apply to this set of objects

Routing – Backshell Termination by Attribute/Property

The “Backshell Termination” custom constraint allows you to control the creation of backshell terminations. Here is the interface:

```
public interface IXBackshellTerminationConstraint extends
IXConstraint
{
     boolean match(IXSignal       signal,
                   IXDoDontResult result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that potentially could have a backshell termination. Based on the information on this signal, you should return true if you need to enforce or prevent backshell termination. In addition, you will need to confirm what type of decision you have made using the result object that was passed in. This result object implements the following interface:

```
public interface IXDoDontResult
{
     enum DoDont {DO, DONT}

     void setValue(DoDont value);

     DoDont getValue();
}
```

By calling the `setValue(DO)` on this result object, you will be indicating that the termination should be created.

By calling the `setValue(DONT)` on this result object, you will be indicating that the termination should NOT be created.

Routing – Cost of Multiterm

Custom Constraint allows custom calculation of the cost of multiterms during routing.

```
public interface IXCostMultitermConstraint
{
      boolean match(IXSignal signal,
              IXCavity cavity,
              IXCostResult result)

      boolean match(IXSignal signal,
              IXConnector inlineConnector,
              IXCostResult result)

}
```

Determine if constraint applies given these objects. If so, allows custom calculation of cost of a cavity with multiple terminations of the signal used during routing of signals

Parameters:
`signal` - - Current signal on the wire
`cavity` - - Cavity to cost
`inlineConnector` - - where a cavity may be created for the signal
`result` - - Use to store the cost value. Set when returning "true" result.

Returns:
boolean - Does the constraint apply to this specific set of objects?

Routing – Cost of Splice

The "Cost of Splice" custom constraint allows you to define rules for calculating the cost of a splice during the routing of signals. Cost is used to determine if one potential route for a signal is better or worse than another.

```
public interface IXCostSpliceConstraint extends IXConstraint
{
     boolean match(IXSignal       signal,
                   IXBundle       bundle,
                   IXCostResult   result);

     boolean match(IXSignal       signal,
                   IXBundleRegion region,
                   IXCostResult   result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that potentially could be routed via a splice. This information can be used, by you, to contribute to the cost calculation of the route.

For one of the methods, the `IXBundle` input parameter allows you to obtain information about the bundle the splice is in. This information can be used, by you, to contribute to the cost calculation of the route.

For the other method, the `IXBundleRegion` input parameter allows you to obtain information about the bundle region the splice is in. This information can be used, by you, to contribute to the cost calculation of the route.

If you wish to determine the cost, the `match()` method must return true. In addition, you should set the cost on the result object. The result object that is passed into the `match()` method uses the following interface:

```
public interface IXCostResult
{
     void setCost(double cost);

     double getCost();
}
```

Use the `setCost()` method to set the cost of placing the splice. The cost is a normalized cost. Cost can be any double and the larger the number the more expensive it is. This is a weighting factor for routing not an actual cost in currency. The default is 1.0 when there is no constraint.

Routing – Cost of Wire per Unit Length

The "Cost of Wire per unit length" custom constraint allows you to determine the cost of routing a wire based on its length. Cost is used to determine if one potential route for a signal is better or worse than another. Here is the interface:

```
public interface IXCostWireConstraint extends IXConstraint
{
     boolean match(IXSignal       signal,
                   IXBundle       bundle,
                   IXCostResult   result);

     boolean match(IXSignal       signal,
                   IXBundleRegion region,
                   IXCostResult   result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that could potentially be routed via the wire. This information will be used, by you, to contribute to the cost calculation of the route.

With respect to the first method, the `IXBundle` input parameter allows you to obtain information about the bundle the splice is in. This information will be used, by you, to contribute to the cost calculation of the route.

With respect to the second method, the `IXBundleRegion` input parameter allows you to obtain information about the bundle region the splice is in. This information will be used, by you, to contribute to the cost calculation of the route.

If you wish to determine the cost, the `match()` method must return true. In addition, you should set the cost on the result object. The result object uses the following interface:

```
public interface IXCostResult
```

```
{
        void setCost(double cost);

        double getCost();
}
```

Use the `setCost()` method to set the cost of the wire per unit length. The unit length is defined in Capital Project. The cost is a normalized cost per unit length. Cost can be any double and the larger the number the more expensive it is. This is a weighting factor for routing not an actual cost in currency. The default is 1.0 when there is no constraint.

Routing – Cost per wire for Inline/Junction Box

Custom Constraint allows custom calculation of the cost of an Inline or Junction Box during routing.

```
public interface IXCostInlineJunctionBoxConstraint
{
      boolean match(IXSignal signal,
                    IXConnector inline,
                    IXCostResult result)

      boolean match(IXSignal signal,
                    IXSlot jb,
                    IXCostResult result)
}
```

Determine if constraint applies given these objects. If so, allows custom calculation of cost of an Inline used during routing of signals

Parameters:
`signal` - - Current signal on the wire
`inline` - - Inline to cost
`jb` - - Junction Box to cost
`result` - - Use to store the cost value. Set when returning "true" result.

Returns:
`boolean` - Does the constraint apply to this specific set of objects?

Routing – Generate center strip splices

Custom Constraint - used to control the conversion of splices to center-stripped. of routed signals in the context of a specific harness level.

```
public interface IXCenterStripSpliceConstraint
{
     boolean match(
          IXSignal signal,
          IXHarnessLevel level,
          java.util.Set<IXWire> wiresInLevel,
          IXCenterStripResult result)
}
```

Determine if constraint applies, for this signal in a specific harness level. This method will be called once for each harness level the signal appears in.

Parameters:
`signal` - - being considered for conversion
`level` - - the current harness level
`wiresInLevel` - - set or wires of the signal in this specific harness level
`result` - - Return object with information regarding conversion

Returns:
`boolean` - Does the constraint apply to this specific set of objects? If false the result object is ignored. This method must return true for all the levels for which it is called in order for the constraint to apply.

Routing – Map to Inline Cavity by Signal/Attribute Property

Custom Constraint - used to control the routing of wires of a given signal.

This constraint is provided to help provide control over which cavity (or cavities) a particular signal (or set of signals) will map to.

If this constraint's match(..) method returns true then the cavity mapping result (IXCavityMappingResult parameter) will be taken into consideration during the route. If the result specifies a DO parameter then this constraint will specify the cavity (or cavities) that are to be preferred when mapping the given signals to inline cavities.

If, however, the result specifies a DON'T value for the result then the cavities contained within the result are restrictions rather than preferences.

```
public interface IXCavityMappingConstraint
{
    boolean match(   IXSignal signal,
                     IXConnector connector,
                     IXCavityMappingResult result)
}
```

Determine if constraint applies given these objects. If so, return a result to indicate the proper action.

Parameters:
`signal` - - To be mapped to cavity
`connector` - - The connector that has the cavities being considered
`result` - - Return a result that describes the cavities to either map or explicitly not map to.

Returns:
`boolean` - Does the constraint apply to this specific set of objects?

Routing – Max Wires per Multiterm

The "Max Wires per Multiterm" custom constraint allows you to develop custom rules for calculating the maximum number of Wires that are allowed for a Multiterm when routing signals. Here is the interface:

```
public interface IXMaxWiresPerMultitermConstraint
          extends IXConstraint
{
     boolean match(IXSignal      signal,
                   IXCavity      cavity,
                   IXCountResult result);

     boolean match(IXSignal                signal,
                   IXBackshellTermination backshellTerm,
                   IXCountResult          result);

     boolean match(IXSignal signal,
                  IXConnector inlineConnector,
                  IXCountResult result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that could be terminated here. The information on this signal can be used, by you, to contribute to the value being calculated.

The `IXCavity` input parameter is present on one of the `match()` methods. If the signal could be terminated on a cavity, this method will be called. The information on this object can be used, by you, to contribute to the value being calculated.

The `IXBackshellTermination` input parameter is present on the other `match()` method. If the signal could be terminated on a backshell termination, this method will be called. The information on this object can be used, by you, to contribute to the value being calculated.

If you wish to provide this maximum limit, the `match()` method must return true. In addition, you should set the maximum on the result object. The result object that is passed into the `match()` method uses the following interface:

```
public interface IXCountResult
{
     void setMaximum(int max);

     int getMaximum();
}
```

By calling the `setMaximum()` method, you can set the maximum value. Using `setMaximum(0)` will disable the use of MultiTerms when routing the signal.

Routing – Max Wires per Splice

The "Max Wires per Splice" custom constraint allows you to define rules for calculating the maximum number of Wires that are allowed for a Splice when routing Signals. Here is the interface you need to implement:

```
public interface IXMaxWiresPerSpliceConstraint extends
IXConstraint
{
     boolean match(IXSignal       signal,
                   IXBundle       bundle,
                   IXCountResult  result);

     boolean match(IXSignal       signal,
                   IXBundleRegion region,
                   IXCountResult  result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that potentially could be routed via the splice. This is the splice that the "maximum number of wires allowed" is being calculated for.

With respect to the first method, the `IXBundle` input parameter allows you to obtain information about the bundle the splice is in. This information will be used, by you, to contribute to the maximum number of wires.

With respect to the second method, the `IXBundleRegion` input parameter allows you to obtain information about the bundle region the splice is in. Again, this information will be used, by you, to contribute to the maximum number of wires.

If you wish to provide this maximum limit, the `match()` method must return true. In addition, you should set the maximum limit on the result object. The result object that is passed into the `match()` method uses the following interface:

```
public interface IXCountResult
{
     void setMaximum(int max);

     int getMaximum();
}
```

By calling the `setMaximum()` method, you can set the maximum value. Using `setMaximum(0)` will disable the use of splices when routing the signal.

Routing – Minimum Splice Separation

The "Minimum Splice Separation" custom constraint allows you to develop rules for calculating the minimum splice separation allowed between a splice and a connector, splice or takeout when routing signals. Here is the interface you need to implement:

```
public interface IXMinSpliceSeparationConstraint extends
IXConstraint
{
     boolean match(IXSignal           signal,
                   IXBundle           bundle,
                   IXSeparationResult result);

     boolean match(IXSignal           signal,
                   IXBundleRegion     region,
                   IXSeparationResult result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that could potentially be routed via the splice. This information will be used, by you, to contribute to the final minimum separation calculation.

For one of the methods, the `IXBundle` input parameter allows you to obtain information about the bundle the splice is in. This information can be used, by you, to contribute to the minimum separation calculation.

For the other method, the `IXBundleRegion` input parameter allows you to obtain information about the bundle region the splice is in. Again, this information can be used, by you, to contribute to the final minimum separation calculation.

If you wish to provide this minimum separation, the `match()` method must return true. In addition, you should set the minimum separation on the result object. The result object that is passed into the `match()` method uses the following interface:

```
public interface IXSeparationResult
{
      enum IXSeparationType {  CONNECTOR, SPLICE, TAKEOUT }

      void setObject(IXSeparationType objectType);

      void setMinDistance(double minDistance);

      IXSeparationType getObject();

      double getMinDistance();
}
```

The `setObject()` method is used to indicate what other object the minimum separation should be applied to (are we calculating the minimum separation between this splice and `CONNECTOR`, this splice and `SPLICE` or this splice and `TAKEOUT`).

The `setMinDistance()` method is used to indicate the minimum separation in units defined in Capital Project.

Routing – Network Specification

Custom Constraint - allows custom specification of the Network Specification constraints for a top-level multicore or signal during routing. The Network Specification constraints are: o the maximum length of the bus between the ends o the minimum length of a stub o the maximum length of a stub o the minimum distance between stub splices

```
public interface IXNetworkSpecificationConstraint extends
IXConstraint
  {
        boolean match(IXMulticore multicore,
                      IXNetworkSpecificationResult result)

        boolean match(IXSignal signal,
                      IXNetworkSpecificationResult result)
  }
```

Determine the network specification (Max length, min/max stub length, min stub distance) for the given top-level multicore

Parameters:
`signal` - - signal to get network specification for
`multicore` - - top-level multicore to get network specification for
`result` - - Use to return the network specification values for the top-level multicore

Returns:
`boolean` - `true` if the constraint applies to the top-level multicore if `false` then the result value is not used

Routing – Pigtails Usage by Attribute/Property

The "Use Pigtails" custom constraint allows you to develop rules for the creation of shield terminations during routing. Here is the interface:

```
public interface IXPigtailUsageConstraint extends
IXConstraint
{
     boolean match(IXSignal       signal,
                   IXDoDontResult result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that could potentially have a shield termination. Based on the information on this signal, you should return true if you need to enforce or prevent shield termination. In addition, you will need to confirm what type of decision you have made using the result object that was passed in. This result object implements the following interface:

```
public interface IXDoDontResult
{
     enum DoDont {DO, DONT}

     void setValue(DoDont value);

     DoDont getValue();
}
```

By calling the `setValue(DO)` on this result object, you will be indicating that the termination should be created.

By calling the `setValue(DONT)` on this result object, you will be indicating that the termination should NOT be created.

Routing – Protect Wiring by Attribute/Property

Custom constraint used to prevent modification of wiring by the following operations

1. Abstract Wiring Synthesis
2. Composite Wiring Synthesis (when both "Clear Promoted Options" and "Delete all Wiring, Harness Levels and Configurations" are unchecked)

3. Generate Option Expressions
4. Synthesize All Wiring
5. Delete All Wiring
6. Generate Center-Strip Splices

Please note that other operation may modify the wiring.

```
public interface IXProtectSignalWiringConstraint
{
    boolean match(IXSignal signal)
}
```

Indicates whether wiring of the specified signal should be protected (not modified by batch synthesis operations).

Parameters:
`signal` - signal to check

Returns:
`true` - if wiring should be protected or false otherwise

Routing – Route by Attribute/Property

This custom constraint allows you to implement generic rules for routing signals. Implementations of this constraint can be used to decide if the signal should or should not be routed based on some property or attribute of the signal. Here is the interface you need to implement:

```
public interface IXRoutingConstraint extends IXConstraint
{
     boolean match(IXSignal         signal,
                   IXBundle         bundle,
                   IXOnlyDontResult result);

     boolean match(IXSignal         signal,
                   IXBundleRegion   region,
                   IXOnlyDontResult result);
}
```

The `IXSignal` input parameter allows you to obtain information about the signal that potentially could be routed via the bundle.

Considering the first method, the `IXBundle` input parameter allows you to obtain information about the bundle the signal could be routed in.

Considering the second method, the `IXBundleRegion` input parameter allows you to obtain information about the bundle region the signal could be routed in.

If you decide that the constraint you are implementing has to be triggered, you should return true. In addition, you will need to indicate if the signal should or should not be routed via the bundle or bundle region. To allow you to do this, a result object is passed in that implements the following interface:

```
public interface IXOnlyDontResult
{
     enum OnlyDont {ONLY, DONT}

     void setValue(OnlyDont value);

     OnlyDont getValue();
}
```

Calling `setValue(ONLY)` on the result object will tell Capital Integrator that the signal should be routed here via the specified bundle or bundle region.

Calling `setValue(DONT)` on the result object will tell Capital Integrator that the signal should be NOT routed here via the specified bundle or bundle region.

Module Assignation – Assign Module Codes for All Design Objects via Plugin

This custom constraint allows control of how a specific set of selected objects, or all objects in the current design should be assigned module codes as part of the Capital ModularXC assignation action:

```
public interface IXModuleAssignationConstraint
               <XMC extends IXModuleCode>
       extends IXConstraint
{
     boolean assignModuleCodes (
          Set<IXModuledObject>           objects,
          IXHarnessDesign                design,
          IXModuleAssignationResult<XMC> result);
}
```

The `assignModuleCodes()` method is the method that does the work. If the user of Capital Modular XC wants your custom plugin to work on a specific set of objects, those objects will be in the `objects` parameter. If the user wants to do assignation on the entire design, the `design` parameter should be used. The result of assignation should be applied to the `result` parameter =>

```
public interface IXModuleAssignationResult
               <XMC extends IXModuleCode>
       extends IXConstraint
{

     IXModuledDesign getModuledDesign();

     void addModuleCodeToObject (
          IXModuledObject object,
          XMC code);

     void addNewNonTechnicalModuleCodeToObject (
          IXModuledObject object,
          String code);

     void addNewTechnicalModuleCodeToObject (
          IXModuledObject object,
          String code);

     void addModuleCodeToCombinationForTechnicalModule (
          String techModCode,
          XMC code);

     boolean setObjectsToBeAssigned(boolean b);

     void addMessage (
          String message);

     void populateUserModuleCodes();

     void removeNonTechnicalCodesFromWires();
     …

}
```

To determine the codes that can be assigned to an object, you need to call methods on the `IXModuledDesign` object returned by `IXModuleAssignationResult.getModuledDesign()`.

Use `IXModuledDesign.getAvailableFunctionalModuleCodes()` for the available functional module codes

or

`IXModuledDesign.getAvailableProductionModuleCodes()` for the available production module codes.

To add an existing module code to an object, you need to use the `IXModuleAssignationResult.addModuleCodeToObject()` method. The `object` parameter is the object you want to add the code to. The `code` parameter is the code you want to add.

To add a completely new module code to an object, use the `IXModuleAssignationResult.addNewNonTechnicalModuleCodeToObject()` method. The `object` parameter is the object you want to add the code to. The `code` parameter is the name of the new code you want to be created and added to the object.

To add a completely new technical module code to an object, use the `IXModuleAssignationResult.addNewTechnicalModuleCodeToObject()` method. The `object` parameter is the object you want to add the code to. The `code` parameter is the name of the new technical code you want to be created and added to the object.

To create a new technical module code with an associated combination, use the `addModuleCodeToCombinationForTechnicalModule()` method. For example, to create a technical code TMX with the combination FM1 and FM2 =>

By default, module codes are assigned to an object's "Generated" module codes. You can alter this default behavior to update an object's "User" module codes by using the following method:
`r.populateUserModuleCodes();`

By default, the assignation process does not remove non-technical module codes from wire objects. You can alter this default behavior by using the following method:

```
result.removeNonTechnicalCodesFromWires();
```

If this method is called, all non-technical module codes will be removed from each wire object during the assignation process. By default, the non-technical codes are removed from the wire's "Generated" module codes, unless `populateUserModuleCodes()` has been called in the plugin, in which case the codes are removed from the wire's "User" module codes.

```
IXModuleAssignationResult r = …
r.addModuleCodeToCombinationForTechnicalModule ("TMX", fm1);
r.addModuleCodeToCombinationForTechnicalModule ("TMX", fm2);
```

Your custom plugin should call `setObjectsToBeAssigned()` with `true` if the changes you have made to the result object should be applied, or `false` if they should not.

Your custom plugin should call `addMessage()` with a `message` string if it needs to report any issues. These will appear in the output window of Capital ModularXC.

Module Assignation – Assign Module Codes for Selected Object(s)

This custom constraint allows control of how a specific object should be assigned module codes as part of the Capital ModularXC assignation action:

```
public interface
       IXModuleAssignationObjectAlgorithmConstraint
     extends IXConstraint
{

     boolean shouldObjectBeAssigned (
          IXModuledObject                object,
          IXModuleAssignationResult<XMC> result);

     boolean assignModuleCodes (
          IXModuledObject object,
          List<IXWire>    wires,
          IXModuleAssignationResult<XMC> result);

}
```

The `shouldObjectBeAssigned()` method is used to decide if this custom constraint can handle this object. The object that the constraint is about to work on is in the `object` parameter. Returning `false` will stop this constraint from working on this object but it will not stop other constraints from trying to work on the object.

The `assignModuleCodes()` method is the method that does the work. The object that the constraint needs to work on is in the `object` parameter. The wires that are touching or passing through that object are in the `wires` parameter. The result of assignation should be applied to the `result` parameter =>

```
public interface IXModuleAssignationResult
               <XMC extends IXModuleCode>
       extends IXConstraint
{

     IXModuledDesign getModuledDesign();

     void addModuleCodeToObject (
          IXModuledObject object,
          XMC code);

     void addNewNonTechnicalModuleCodeToObject (
          IXModuledObject object,
          String code);
```

```
        boolean setObjectsToBeAssigned(boolean b);

        void addMessage (
             String message);

        …

    }
```

To determine the codes that can be assigned to an object, you need to call methods on the `IXModuledDesign` object returned by `IXModuleAssignationResult.getModuledDesign()`.

Use `IXModuledDesign.getAvailableFunctionalModuleCodes()` for the available functional module codes

or

`IXModuledDesign.getAvailableProductionModuleCodes()` for the available production module codes.

To add an existing module code to an object, you need to use the `IXModuleAssignationResult.addModuleCodeToObject()` method. The `object` parameter is the object you want to add the code to. The `code` parameter is the code you want to add.

To add a completely new module code to an object, use the `IXModuleAssignationResult.addNewNonTechnicalModuleCodeToObject()` method. The `object` parameter is the object you want to add the code to. The `code` parameter is the new code you want to be created and added to the object.

Your custom plugin should call `setObjectsToBeAssigned()` with `true` if the changes you have made to the result object should be applied, or `false` if they should not.

Your custom plugin should call `addMessage()` with a `message` string if it needs to report any issues. These will appear in the output window of Capital ModularXC.

Module Assignation – Assign Technical Module Codes for Selected Object(s)

This custom constraint allows control of how a specific object should be assigned technical module codes as part of the Capital ModularXC assignation action:

```
    public interface IXModuleAssignationTechnicalConstraint
           extends IXConstraint
```

```
{
     boolean shouldObjectBeAssigned (
          IXModuledObject                 object,
          IXModuleAssignationResult<XMC> result);

     boolean assignModuleCodes (
          IXModuledObject object,
          List<IXWire>    wires,
          IXModuleAssignationResult<XMC> result);
}
```

The `shouldObjectBeAssigned()` method is used to decide if this custom constraint can handle this object. The object that the constraint is about to work on is in the `object` parameter. Returning `false` will stop this constraint from working on this object but it will not stop other constraints from trying to work on the object.

The `assignModuleCodes()` method is the method that does the work. The object that the constraint needs to work on is in the `object` parameter. The wires that are touching or passing through that object are in the `wires` parameter. The result of assignation should be applied to the `result` parameter =>

```
public interface IXModuleAssignationResult
               <XMC extends IXModuleCode>
       extends IXConstraint
{

     IXModuledDesign getModuledDesign();

     void addNewTechnicalModuleCodeToObject (
          IXModuledObject object,
          String code);

     void addModuleCodeToCombinationForTechnicalModule (
          String techModCode,
          XMC code);

     boolean setObjectsToBeAssigned(boolean b);

     void addMessage (
          String message);

     …

}
```

To determine the codes that can be assigned to an object, you need to call methods on the `IXModuledDesign` object returned by `IXModuleAssignationResult.getModuledDesign()`.

Use `IXModuledDesign.getAvailableFunctionalModuleCodes()` for the available functional module codes

or

`IXModuledDesign.getAvailableProductionModuleCodes()` for the available production module codes.

To add a completely new technical module code to an object, use the `addNewTechnicalModuleCodeToObject()` method. The `object` parameter is the object you want to add the code to. The `code` parameter is the new technical code you want to be created and added to the object.

To create a new technical module code with an associated combination, use the `addModuleCodeToCombinationForTechnicalModule()` method. For example, to create a technical code TMX with the combination FM1 and FM2

```
IXModuleAssignationResult r = …
r.addModuleCodeToCombinationForTechnicalModule ("TMX", fm1);
r.addModuleCodeToCombinationForTechnicalModule ("TMX", fm2);
```

Your custom plugin should call `setObjectsToBeAssigned()` with `true` if the changes you have made to the result object should be applied, or `false` if they should not.

Your custom plugin should call `addMessage()` with a `message` string if it needs to report any issues. These will appear in the output window of Capital ModularXC.

Cavity Details Generation – Set Cavity Details Generation Method

This custom constraint is to define the method to be used for generating wire combinations (hence, cavity details) for a cavity, on a composite harness:

```
public interface IXCavityDetailGeneratorConstraint
     extends IXConstraint
{
     void generate(
          Collection<? extends IXWire> wiresAtCavity,
          IXCombinationGeneratorResult result);
```

```
}
```

The `generate()` method is to be implemented to identify the set of combinations to be generated. Filling in the `result` object with those combinations (sets) of wires, that could be any mix of wires from the original set (`wiresAtCavity`) passed to the `generate` method.

```
public interface IXCombinationGeneratorResult
{

    void addCombination(Set<? extends IXWire> combination);

    void setReasonForFailure(String reason);


    Set<Set<? extends IXWire>> getCombinations();

    String getReasonForFailure();
}
```

The result object provides a method through which a combination could be added, using `addCombination`. The set of wires passed to `addCombination` should be a subset of, or the same as the original set of wires: `wiresAtCavity`.

The result object provides a way to invalidate itself, using `setReasonForFailure`. That could be done through passing a non-null string value to this method, which would flag the result object as invalid.
If this method isn't called, or called with a null value, that will flag the result object as valid.

The string message set using this method is then displayed at the Cavity Component Management dialog.

Exercise

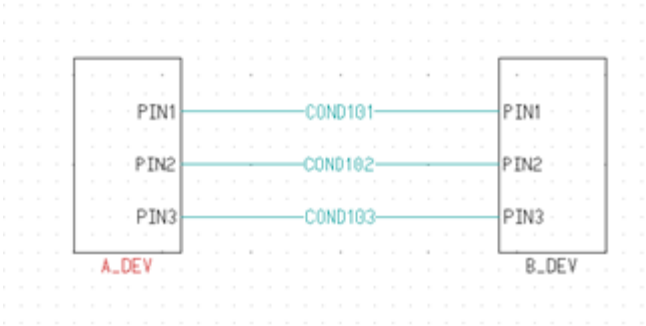
In this exercise, you will develop a custom constraint for the placement of devices in a Java plugin. Here is the specification of this custom constraint: it should place all devices starting with `A_` into slot `A` and all devices starting with `B_` into slot `B`.

You need to be familiar with Capital Integrator to do this exercise.

1) First, you should start by creating some test data.

    a. Start Capital Design

    b. Create a new Capital Logic design. There should be two devices called `A_DEV` and `B_DEV`. Ensure that there are some nets connecting them. Here is an example of what you should have:



    c. Create a new Capital Integrator design. There should be two slots called `A` and `B`. Ensure that there is a bundle that connects them. Here is an example of what you should have:

    d. Associate the logical design with the integrator design using the Actions -> Associate Logical Designs… menu.

    e. Save your new designs. Do not exit the Capital Design application however.

2) The next step is to create a new custom placement plugin from a template.

a. Ensure that your Java development environment has been set-up. If you have completed the Java plugin exercise then it should already be setup. If you have not, then you should stop this exercise and complete it now.

b. Open a UNIX/DOS console. Ensure that the environment variables required for Java, ANT and Capital have been setup in this console. Cd into the following folder within the Capital installation:

```
doc/plugin/examples/Java
```

c. In the above folder, there is a template placement constraint. Here is the location of the file as seen from the Capital installation folder:

```
doc/plugin/examples/Java/TemplatePlacementConstraint.java
```

Create a new folder in the following location within the Capital installation folder:

```
doc/plugin/examples/Java/src/exercise
```

The folder `src` is the directory where all the example Java plugin source code has been placed. This should exist already. The new folder `exercise` is the package in which your plugin will reside.

d. Copy the template into the new folder you have just created and change its name to `NamePlacementConstraint.java`. The result should be a file in the following location:

```
doc/plugin/examples/Java/src/exercise/NamePlacementConstrain
t.java
```

e. Edit this new file.

f. Ensure the plugin is in the correct package. This can be done by adding the following line at the start of the file (before the imports):

```
package exercise;
```

If you use a template as a starting point for your new plugin, you must remember to specify the package it is in.

g. You also need to ensure that the name of the class matches the name of the file. You will need to replace `TemplatePlacementConstraint` with `NamePlacementConstraint` in this file. There are two places where this needs to be done. One is the class definition, the other is the constructor.

h. Change the name returned by the plugin by changing the string returned by the `getName()` method:

```
public String getName()
{
     return "Name-based Device Placement Constraint";
}
```

i. Change the description returned by the plugin by changing the string returned by the `getDescription()` method:

```
public String getDescription()
{
     return "Constraint for the name-based device
placement";
}
```

j. Remember to save the file when these changes are completed.

k. Back at the UNIX/DOS console, ensure that you are in the following folder:

```
doc/plugin/examples/Java
```

l. Build your new plugin by typing the following command:

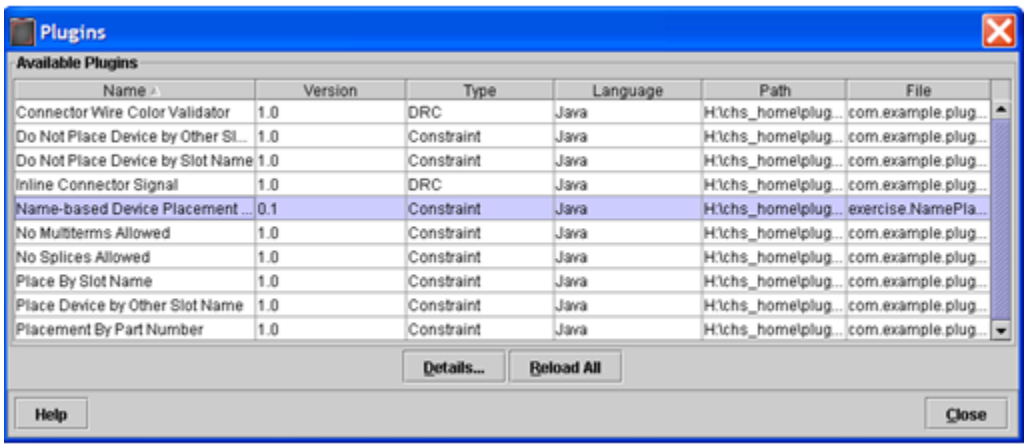
```
ant
```

m. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check the environment variables or check for (and fix) errors in the `NamePlacementConstraint.java` file and try again.

n. Go back to Capital Design.

o. From the main menu, click on the File -> Plugins… menu

p. Press the [Reload All] button on the Plugins dialog. You will see your new plugin appear in the list with the other available plugins. See the selected row in the image below:



Leave this dialog open for now.

3) At this point you have a new plugin. However, it has not been modified to implement the placement specification. This is your next objective.

   a. Edit the `NamePlacementConstraint.java` file.

   b. Edit the match method in the file and add the following code:

```
public boolean match(
     IXDevice        device,
     IXSlot          slot,
     IXDoDontResult result)
{
     // This is the attribute we are looking for.
     final String ATTRIBUTE_NAME = "NAME";

     // If we return true, we indicate that that the
device
     // should be placed in the given slot
     result.setValue(IXDoDontResult.DoDont.DO);

     // Get the name of the slot.
     final String slotName =
slot.getAttribute(ATTRIBUTE_NAME);

     // Get the name of the device.
     final String devName =
device.getAttribute(ATTRIBUTE_NAME);

     // These messages should appear in the application
log file.
     System.err.println("HERE slotName = " + slotName);
```

```
            System.err.println("HERE devName  = " + devName);

            // Test for slot A placement
            if(slotName.startsWith("A") &&
devName.startsWith("A_"))
            {
                 // Place the device in slot A
                 System.err.println("A - OK");
                 return true;
            }

            // Test for slot B placement
            if(slotName.startsWith("B") &&
devName.startsWith("B_"))
            {
                 // Place the device in slot B
                 System.err.println("B - OK");
                 return true;
            }

            // If we get here, we are not going to place the
device
            // in the slot.
            System.err.println("not OK");
            return false;
       }
```

Note the use of the `getAttribute()` method call to get the name of the slot and the device.

c. Change the version number of the plugin, by editing the getVersion() method:

```
public String getVersion()
{
     return "0.2";
}
```

d. Save the file.

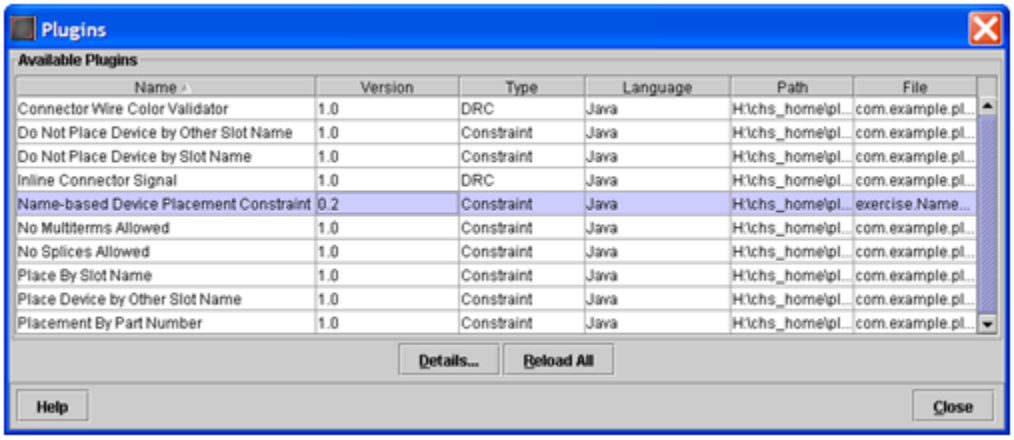
e. Rebuild your plugin by typing the following command in the DOS/UNIX console:

```
ant
```

f. If everything is fine, you will eventually get a “BUILD SUCCESSFUL” message. If you do not, you will need to check for (and fix) errors in the `NamePlacementConstraint.java` file and try again.

g. Go back to Capital Design.

h. Press the [Reload All] button on the Plugins dialog. You will see the version number of your new plugin change:



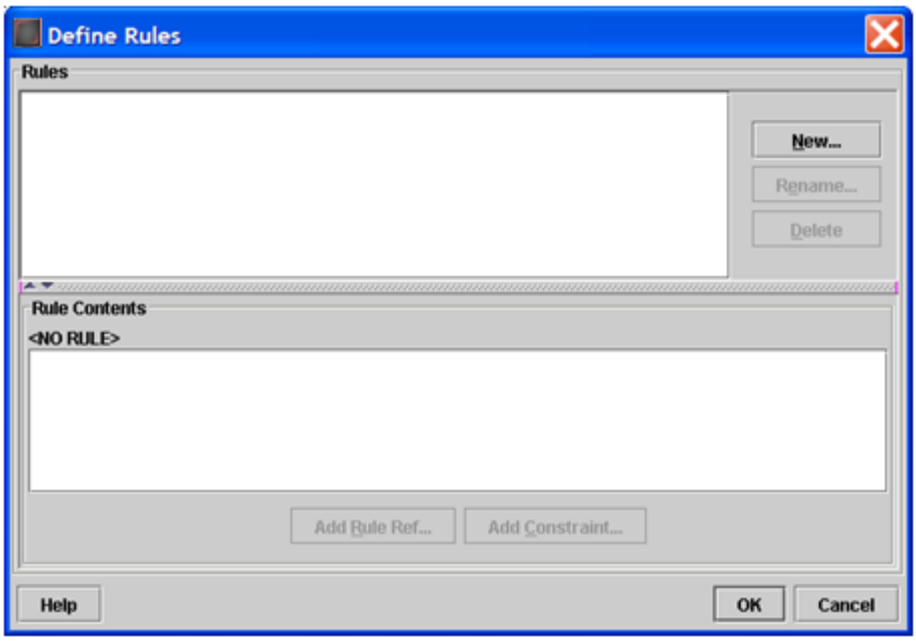
Name	Version	Type	Language	Path	File
Connector Wire Color Validator	1.0	DRC	Java	H:\chs_home\pl...	com.example.pl...
Do Not Place Device by Other Slot Name	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...
Do Not Place Device by Slot Name	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...
Inline Connector Signal	1.0	DRC	Java	H:\chs_home\pl...	com.example.pl...
Name-based Device Placement Constraint	0.2	Constraint	Java	H:\chs_home\pl...	exercise.Name...
No Multiterms Allowed	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...
No Splices Allowed	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...
Place By Slot Name	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...
Place Device by Other Slot Name	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...
Placement By Part Number	1.0	Constraint	Java	H:\chs_home\pl...	com.example.pl...

The change in version number confirms that your new plugin has been loaded.

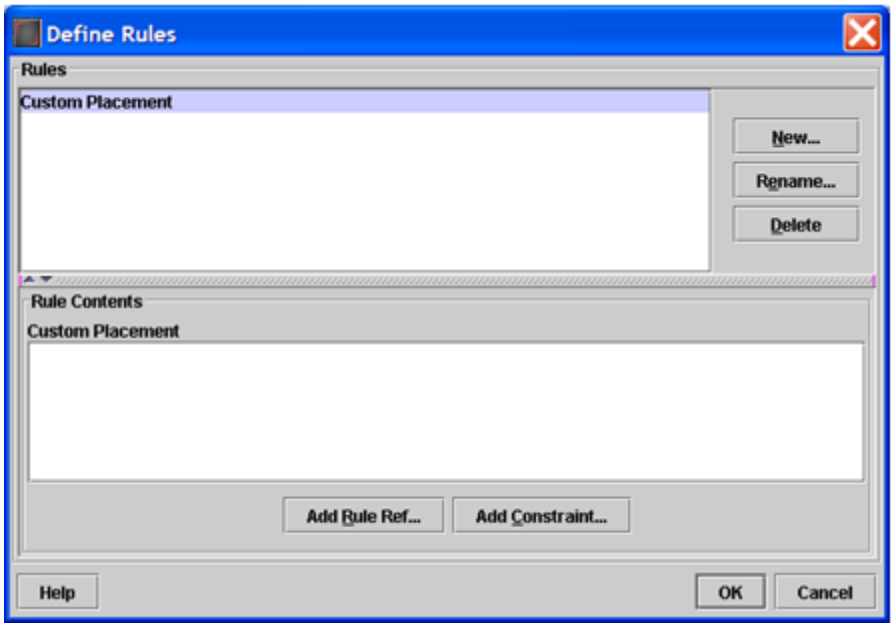
i. Close the Plugins dialog.

4) At this point, you are in a position to try your new plugin.

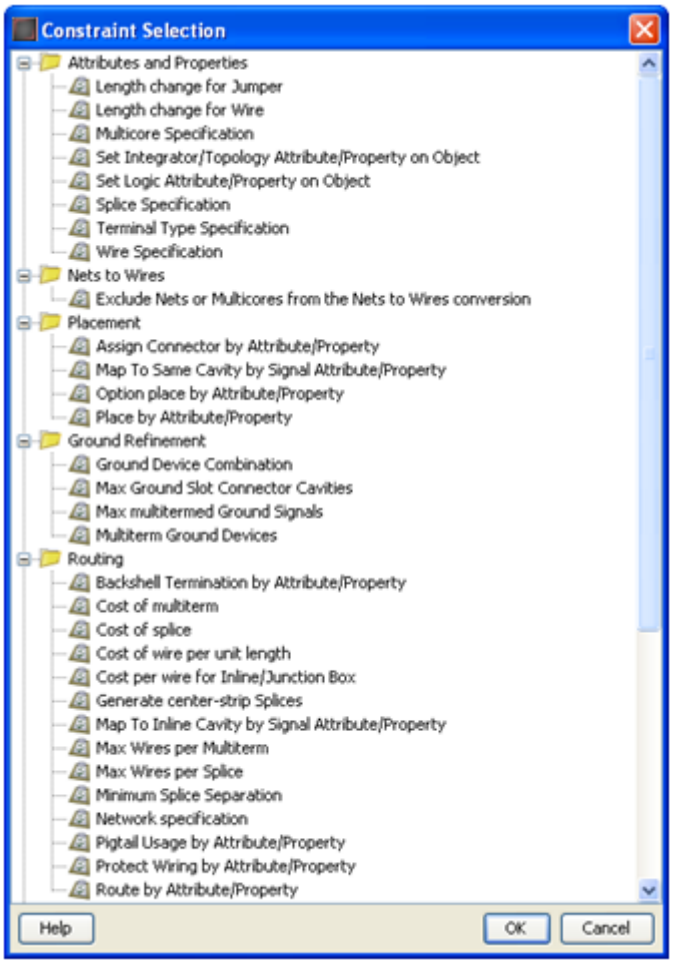
a. On the main menu of Capital Design, click the Define -> Rules menu. This will open the following window:



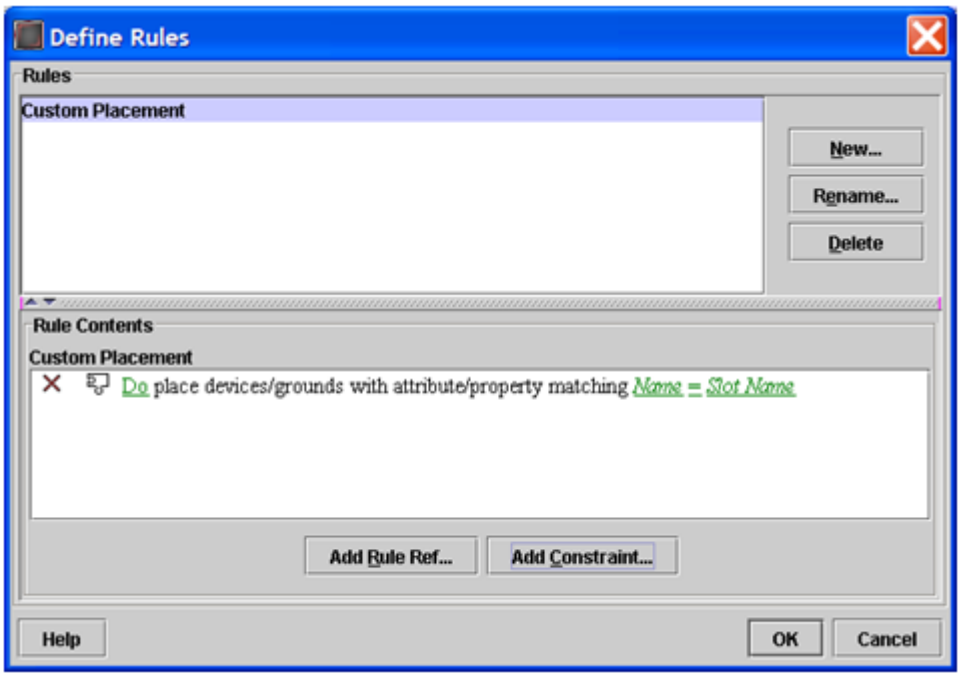
b. Press the [NEW] button to create a new rule. Call the rule "Custom Placement" and click [OK]. Your Define Rules dialog will now look like this:



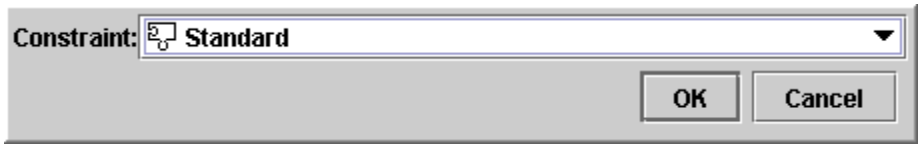
c. With the "Custom Placement" rule still selected, click on the [Add Constraint…] button to add a new constraint. This will open the following window:



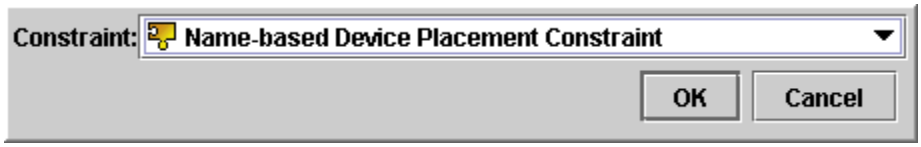
d. Select the Place by Attribute/Property option and click [OK]. The “Define Rules” dialog will now look like the following:



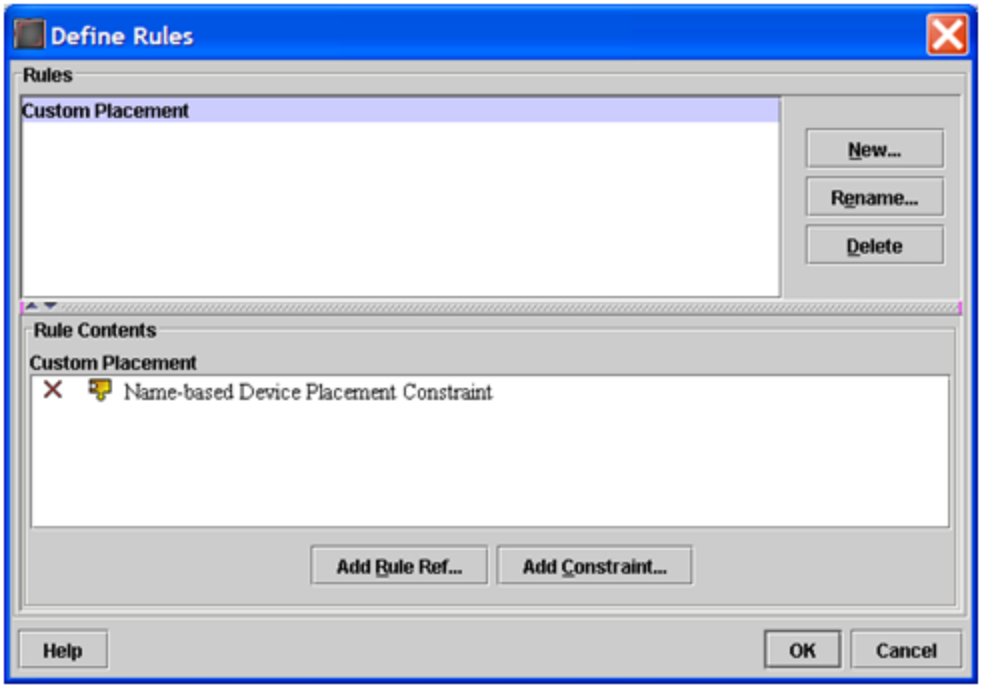
e. At this point, the standard (built-in) rule for placement is selected. You need to replace this with your custom rule. To do this, click on the icon that looks like a puzzle piece. The following window will open confirming that you are currently using the standard rule:



f. Use the drop-down selection to select your new placement rule (note that the name used for this new rule is the name returned by the `getName()` method for the plugin):



g. Click on the [OK] button to confirm your selection. Your Design Rule dialog should now show the following:

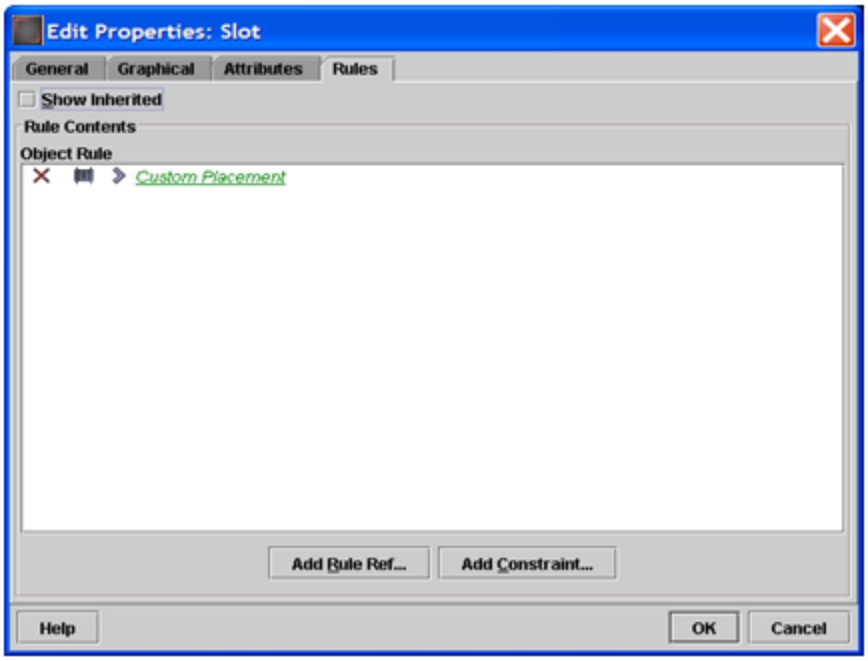


At this point, your new custom rule is selected.

h. Click [OK] to close the Design Rule window.

i. For each slot on the Integrator plane, you need to open the properties for the slot and ensure that your new “Custom Placement” rule is being used for that slot.

When the properties window for the slot is open, click on the [Add Rule Ref…] button, select the “Custom Placement” rule and click [OK]. The properties window for the slot will now look like this:



j. Ensure that you click [OK] to save the changes to the slot properties. Ensure also that you apply the "Custom Placement" rule to both slots.

k. From the main menu of Capital Design, click the Place -> Auto Place All Devices menu. You should see the result of the placement in the output window:

You should now look at the log file for Capital Design. In this file you will see the messages that were printed from the plugin, for example, `System.err.println("A - OK");`. Any use of `System.out` and `System.err` will be directed to the log file of the application you are running. This is useful for debugging.

Once you have debugged your plugin, however, you should remove or disable your debug messages so that performance is not hindered.

At this point, you should know the following:

- How to iteratively develop Java Custom constraints.
- How to interrogate the business objects in Java.

Guidance

Always develop custom constraints in Java for performance. Also ensure that any debug messages are removed or disabled before your custom constraint is deployed.

If you have developed your own custom constraints, and they are in use at your company, care should be taken when maintaining those constraints. You should avoid renaming the class or moving the class to another package. If you do this, any previously created design will no longer be able to find the custom constraint if you deploy an updated plugin that has a new class name or package. Any custom constraint used in a design will have a reference to that constraint stored with that design. This reference will include the class and package name for that constraint.

Examples

At this point, you should have a good idea how to implement this type of plugin. Some examples have been provided to help you. These examples can be found in the following location within the Capital installation:

```
doc/plugin/examples/Java/src/com/example/plugin/constraint
```

The following examples are good starting points for Capital Integrator:

- `DoNotPlaceBySlotName.java` – this example shows how to create a rule that prevents device placement by name.

- `NoSplicesAllowed.java` – this example shows how to create a rule that prevents splice creation.

The following examples are good starting points for Capital ModularXC:

- `FunctionalPropertyToCodeAssignationConstraint.java` – this example performs assignation by converting a string property on an object into a module code.

- `ProductionPropertyToCodeAssignationConstraint.java` – this example performs assignation by converting a string property on an object into a module code.

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).